3640. Trionic Array II

Solved

Hard | Topics | Companies | Hint

You are given an integer array `nums` of length `n`.

A **trionic subarray** is a contiguous subarray `nums[l...r]` (with `0 <= l < r < n`) for which there exist indices `l < p < q < r` such that:

- `nums[l...p]` is **strictly** increasing,
- `nums[p...q]` is **strictly** decreasing,
- `nums[q...r]` is **strictly** increasing.

Return the **maximum** sum of any trionic subarray in `nums`.

Example 1:

Input: nums = [0,-2,-1,-3,0,2,-1]

Output: -4

Explanation:

Pick `l = 1`, `p = 2`, `q = 3`, `r = 5`:

- `nums[l...p] = nums[1...2] = [-2, -1]` is strictly increasing (`-2 < -1`).
- `nums[p...q] = nums[2...3] = [-1, -3]` is strictly decreasing (`-1 > -3`)
- `nums[q...r] = nums[3...5] = [-3, 0, 2]` is strictly increasing (`-3 < 0 < 2`).
- Sum = `(-2) + (-1) + (-3) + 0 + 2 = -4`.

Example 2:

> **Input:** nums = [1,4,2,7]
>
> **Output:** 14
>
> **Explanation:**
>
> Pick `l = 0`, `p = 1`, `q = 2`, `r = 3`:
>
> - `nums[l...p] = nums[0...1] = [1, 4]` is strictly increasing (`1 < 4`).
> - `nums[p...q] = nums[1...2] = [4, 2]` is strictly decreasing (`4 > 2`).
> - `nums[q...r] = nums[2...3] = [2, 7]` is strictly increasing (`2 < 7`).
> - Sum = `1 + 4 + 2 + 7 = 14`.

Constraints:

- $4 <= n = nums.length <= 10^5$
- $-10^9 <= nums[i] <= 10^9$
- It is guaranteed that at least one trionic subarray exists.

Python:

```
class Solution:
    def maxSumTrionic(self, nums: List[int]) -> int:
        n = len(nums)
        res = -float('inf')
        i = 1
        while i < n - 2:
            a = b = i
            net = nums[a]
            while b + 1 < n and nums[b + 1] < nums[b]:
                net += nums[b + 1]
                b += 1
```

```
            if b == a:
                i += 1
                continue

            c = b
            left = right = 0
            lx = rx = -float('inf')

            while a - 1 >= 0 and nums[a - 1] < nums[a]:
                left += nums[a - 1]
                lx = max(lx, left)
                a -= 1
            if a == i:
                i += 1
                continue

            while b + 1 < n and nums[b + 1] > nums[b]:
                right += nums[b + 1]
                rx = max(rx, right)
                b += 1
            if b == c:
                i += 1
                continue

            res = max(res, lx + rx + net)
            i = b
        return res if res != -float('inf') else 0
```

JavaScript:

```
/**
 * @param {number[]} nums
 * @return {number}
 */
var maxSumTrionic = function(nums) {
    const n = nums.length;
    let res = -Infinity;

    for (let i = 1; i < n - 2; ) {
        let a = i, b = i;
        let net = BigInt(nums[a]);

        // Middle decreasing part
        while (b + 1 < n && nums[b + 1] < nums[b]) {
            net += BigInt(nums[++b]);
```

```
        }
        if (b === a) { i++; continue; }

        let c = b;
        let left = 0n, right = 0n;
        let lx = -BigInt(1e16), rx = -BigInt(1e16);

        // Left increasing part
        while (a - 1 >= 0 && nums[a - 1] < nums[a]) {
            left += BigInt(nums[--a]);
            if (left > lx) lx = left;
        }
        if (a === i) { i++; continue; }

        // Right increasing part
        while (b + 1 < n && nums[b + 1] > nums[b]) {
            right += BigInt(nums[++b]);
            if (right > rx) rx = right;
        }
        if (b === c) { i++; continue; }

        let currentSum = Number(lx + rx + net);
        if (currentSum > res) res = currentSum;

        i = b;
    }
    return res;
};
```

Java:

```
class Solution {

    public long maxSumTrionic(int[] nums) {

        int n = nums.length;
        long res= -1 * (long)1e16;

        for(int i=1;i<n-2;i++){

            int a = i;
            int b = i;

            long net = nums[a];
```

```
            while(b+1<n && nums[b+1] < nums[b]){
                net+=(long)nums[b+1];
                b++;
            }

            if(b==a)continue;

            int c= b;

            long left = 0;
            long right = 0;

            long lx =Integer.MIN_VALUE;
            long rx =Integer.MIN_VALUE;

            while(a-1>=0 && nums[a-1] < nums[a]){
                left+=(long)nums[a-1];
                lx = Math.max(lx, left);
                a--;
            }

            if(a==i)continue;

            while(b+1<n && nums[b+1] > nums[b]){
                right+=(long)nums[b+1];
                rx = Math.max(rx, right);
                b++;
            }

            if(b==c)continue;
            i=b-1;
            res = Math.max( res, lx+rx+net);

        }
        return res;
    }
}
```